AGENCY PLATFORM · CONCEPT EXPLAINED

GUID v4 vs v7

What is inside each kind of GUID, how they behave in a PostgreSQL index (measured, not guessed), and what this means for the Agency Platform's primary keys.

IN ONE SENTENCE

Both are 128-bit identifiers stored in a PostgreSQL `uuid` column. **v4 is entirely random. v7 starts with a millisecond timestamp**, so new IDs are always larger than older ones, and that makes database indexes much happier.

1. Real values (generated with .NET 10 on the development machine)

v4 · `Guid.NewGuid()`	**v7** · `Guid.CreateVersion7()` **(2 ms apart)**
`8eb0d4fd-246b-4a4f-83af-7485c09a09e1`	`01a12feb-a5cf-7450-a4ef-1cb7bff9ede6`
`439754d9-080c-4598-9b2f-f936b2acb5a5`	`01a12feb-a5d1-77b1-ba64-0b2678324208`
`fd2106a4-8a03-4911-a99c-4f50628d6216`	`01a12feb-a5d3-71bf-ad62-5ca1de81c785`
`2111d1f4-edbd-4cd0-8c8e-3312380082cc`	`01a12feb-a5d6-7cae-98ec-ab79776a31e2`
`03672a48-63bb-4039-9b3e-3d562f113417`	`01a12feb-a5d8-71f2-a59c-76957004814e`
Already sorted? **No**	Already sorted? **Yes**

The v4 values have no pattern; the v7 values all start with `01a12feb-a5…` and climb. The 13th character is the **version**: `4` or `7`.

2. What is inside

v4 `8eb0d4fd-246b-4a4f-83af-7485c09a09e1`

122 random bits (everything except the version and variant digits)

4 / 7 = version digit
8, 9, a, b = variant digit (RFC 9562)

v7 `01a12feb-a5cf-7450-a4ef-1cb7bff9ede6`

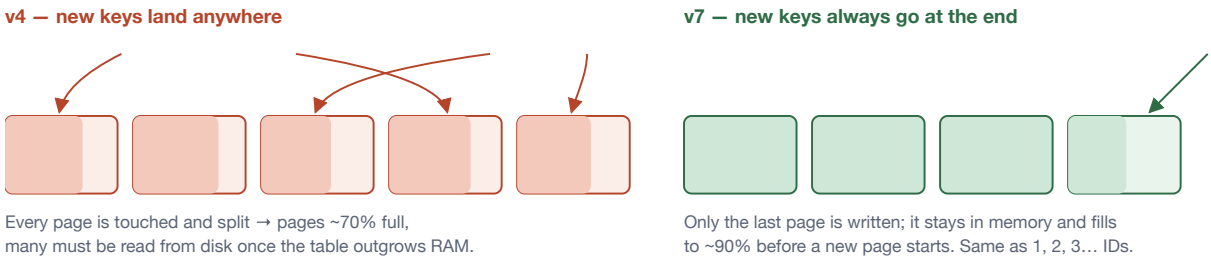
48-bit timestamp (ms) | 74 random bits

01a12feba5cf (hex)
= 1 791 805 334 991 ms since 1970
= 2026-10-12 11:42:14.991 UTC

PostgreSQL decodes it too: uuid_extract_timestamp(id)

3. Why the order matters: how a B-tree index grows

A primary key is stored in a **B-tree index**: a sorted structure split into 8 KB pages, like a phone book. Where a new key lands decides how much work an insert costs.



4. Measured on PostgreSQL 18: 1,000,000 rows each

Two tables with a `uuid PRIMARY KEY`, filled with `gen_random_uuid()` (v4) and `uuidv7()` (v7) in a throwaway Docker container.

	v4 (random)	v7 (time-ordered)
Insert time	1.77 s	**1.22 s** (~30% faster)
Primary-key index size	38 MB	**30 MB** (~20% smaller)
Index page fill level	71%	**90%**
Leaf fragmentation	49.8%	**0%**
`ORDER BY id DESC LIMIT 3`	rows 888135, 86808, 465743 (random)	rows 1000000, 999999, 999998 (newest)

READ THIS RESULT CAREFULLY

This test is small: the whole table fit in memory. On a table larger than RAM, such as bookings, invoice lines or audit log after a few years, v4's random page visits turn into disk reads, and the gap grows much wider.

5. Side-by-side summary

	v4	v7
Content	122 random bits	48-bit ms timestamp + 74 random bits
.NET	`Guid.NewGuid()`	`Guid.CreateVersion7()` (.NET 9+)
PostgreSQL	`gen_random_uuid()`	`uuidv7()` (PostgreSQL 18+)
Sortable by creation time	❌	✅ to the millisecond
Index-friendly inserts	❌ splits pages, fragments	✅ appends at the end
Can be guessed or counted	❌ no	❌ no (74 random bits per millisecond)
Reveals when it was created	❌ no	⚠️ **yes**, anyone can decode the time
Collision risk	negligible	negligible
Standard	RFC 9562 (formerly RFC 4122)	RFC 9562 (2024)

6. What this means for the Agency Platform

RECOMMENDATION · HANDBOOK DECISION 6

Use v7 as the default for primary keys. The measurements support it.

1. **The ID exists before saving.** `City.Create(...)` sets `Id = Guid.CreateVersion7()` inside the domain, so the use case can return the ID immediately, with no database round trip.
2. **The tables that will grow big benefit most.** Bookings, service lines, ledger entries and audit log are append-heavy, which is exactly v7's best case.
3. "**Newest first**" **comes almost free.** `ORDER BY id DESC` already means newest first.

```
// src/Agency.Domain/Catalogue/City.cs
public static Result<City> Create(string? name)
{
    ...
    return new City { Id = Guid.CreateVersion7(), Name = trimmed };   // v7, set by the domain
}
```

CORRECTION TO THE ADR 0007 EXAMPLE

The ADR example in *ADRs explained* says "partners cannot guess or count IDs". That is still true for v7. It left out one thing: a v7 ID shows its **creation time to the millisecond**. For a booking ID shown to a partner, that is harmless, since the partner knows when they booked. If a future ID's creation time is ever sensitive (for example a public link where the timing gives something away), use v4 for that one entity. Add this as a line under "Consequences" when you write the real ADR.

7. Two gotchas

GOTCHA · SAME MILLISECOND

Two GUIDs that .NET creates in the same millisecond are not guaranteed to be in order relative to each other: their tails are random. This does not matter for indexes or uniqueness. Never use the ID as a precise sequence number. Invoice numbers have their own number series inside a database transaction, as `CLAUDE.md` requires.

GOTCHA · THE DATABASE MATTERS

v7 sorts correctly in PostgreSQL because `uuid` compares byte by byte. SQL Server orders its `uniqueidentifier` type differently, so v7's advantage does not carry over there. That does not affect this project (ADR 0002: PostgreSQL only), but keep it in mind when reading SQL Server advice online.

8. Try it yourself

```
// guids.cs — run with: dotnet run guids.cs   (.NET 10 runs single files directly)
var v4 = Guid.NewGuid();
var v7 = Guid.CreateVersion7();
Console.WriteLine($"{v4}  version={v4.Version}");
Console.WriteLine($"{v7}  version={v7.Version}");

var ms = Convert.ToInt64(v7.ToString("N")[..12], 16);       // first 12 hex digits = timestamp
Console.WriteLine(DateTimeOffset.FromUnixTimeMilliseconds(ms));
```

```
-- in psql against a PostgreSQL 18 container
SELECT gen_random_uuid() AS v4, uuidv7() AS v7;
SELECT uuid_extract_timestamp(uuidv7());
```